

Sorting Programs

Detailed Explanation

An intermediate undergraduate CSE study guide

Eight self-contained C++ programs, explained through their actual code, worked examples, measured results and lab-viva questions.

Comparison experiments	Timing experiments
Insertion, merge, quick and heap sort. Sizes 30 to 1,000, step 10; ten random instances per size.	Insertion sort: 15,000 elements in best, average and worst cases. Radix, bucket and counting sort: 25 elements each.

Important: Insertion sort follows the supplied lab example: its counter records successful shifts only. This guide keeps that convention explicit throughout.

Source code and data: [sorting-algorithms-cpp on GitHub](#)

The explanations match the corrected source at commit 14b5fd3. Recorded timings come from the included CSV files; they are measurements from one machine, not performance guarantees.

Contents

Read the first two chapters before the algorithm chapters. The complete listings in the appendix use the exact source files that produced the recorded results.

1. Reading the C++ programs	3
2. The comparison experiment in main()	5
3. Program 1: Insertion sort	7
4. Program 2: Merge sort	9
5. Program 3: Quick sort	11
6. Program 4: Heap sort	13
7. Program 5: Insertion sort timing	15
8. Program 6a: Radix sort	17
9. Program 6b: Bucket sort	19
10. Program 6c: Counting sort	21
11. Timing, results and graph interpretation	23
12. Revision sheet and running the programs	26
Appendix A. Complete C++ source listings	28
Appendix B. Sources and measurement notes	44

1. Reading the C++ programs

What a sorting program must do

A sorting algorithm rearranges the input values into an order, here ascending. A correct result must satisfy two conditions: it is ordered, and it contains exactly the original values with the original multiplicities. Merely checking that adjacent values are ordered would not catch a lost or duplicated value.

Arrays, indices and parameters

Code or term	Meaning in these programs
<code>int arr[]</code>	An integer-array parameter. The function rearranges the caller's array directly.
<code>n</code>	The number of elements to process. Valid array indices are 0 through $n - 1$.
<code>int &comparisons</code>	A reference parameter: increments change the counter that <code>main()</code> passed in.
<code>key / pivot</code>	A saved value used while inserting an element or partitioning a range.
<code>left, right / low, high</code>	Inclusive endpoints of the range processed by a recursive call.
<code>new int[n] / delete[]</code>	Allocate an array whose length is known at runtime, then release it.
<code>vector<double></code>	A growable bucket list. Its <code>push_back()</code> adds a value and <code>size()</code> gives its length.

A reference changes the original counter

```
int comparisons = 0;
insertionSort(arr, size, comparisons);
// comparisons now contains the count recorded by insertionSort.
```

The ampersand in `int &comparisons` means the function uses the original variable. Without the reference, an ordinary `int` parameter would be a copy, so changes would not update `main()`'s counter.

Short-circuit evaluation protects the array

```
while (j >= 0 && arr[j] > key)
```

C++ evaluates the left side of `&&` first. If `j` is negative, the second condition is not evaluated. This prevents access to `arr[-1]`. Moving the array access before the bounds condition could make the program invalid.

Fixed arrays and standard C++

The comparison experiments declare `arr[1000]` and use only the first `size` entries. The timing demonstration uses `const int n = 15000`, so `arr[n]` has a compile-time constant bound. The supplied `int arr[size]`, with a runtime size, is a compiler extension rather than standard C++.

Supporting headers and test guard

Header / construct	Purpose
<code>iostream</code>	<code>cout</code> prints results; <code>cerr</code> reports errors.
<code>cstdlib</code>	<code>rand()</code> and <code>srand()</code> generate the lab inputs.
<code>cmath</code>	<code>log2()</code> , <code>log10()</code> and <code>isfinite()</code> .
<code>fstream</code> / <code>iomanip</code>	Write result files and choose decimal precision.
<code>algorithm</code>	<code>sort()</code> , <code>equal()</code> and <code>swap()</code> . Library sort is used only as a reference check.
<code>chrono</code> / <code>filesystem</code>	Measure elapsed time and create the results directory; compile with C++17.
<code>#ifndef SORTING_TEST</code>	Include <code>main()</code> in normal builds; exclude it when tests include several source files.

2. The comparison experiment in main()

One experiment, four sorting functions

The first four files have the same experimental structure. Only the sorting function called inside the trial loop changes. Each run tests 98 sizes: 30, 40, 50, ..., 1,000. For each size it generates ten fresh arrays, sorts each, records its counter and reports the arithmetic mean.

```
for (int size = 30; size <= 1000; size += 10) {
    int totalComparisons = 0;
    for (int instance = 0; instance < 10; instance++) {
        int arr[1000], expected[1000];
        for (int i = 0; i < size; i++) {
            arr[i] = rand() % 1000;
            expected[i] = arr[i];
        }
        sort(expected, expected + size);
        int comparisons = 0;
        insertionSort(arr, size, comparisons);
        // Verify arr against expected, then save the trial count.
        totalComparisons += comparisons;
    }
    double averageComparisons =
        static_cast<double>(totalComparisons) / 10.0;
    cout << size << " " << averageComparisons << endl;
}
```

This teaching excerpt omits file-writing and error handling; the appendix contains the complete source. Merge, quick and heap call `mergeSort`, `quickSort` and `heapSort` respectively at the same point.

Why each variable is reset where it is

- **totalComparisons** is reset once per size, so it sums only that size's ten trials.
- **comparisons** is reset once per instance, so every sort starts with a zero counter.
- **arr** is regenerated for every instance. Sorting an already sorted result again would change the experiment.
- **expected** is a copy sorted by the library solely to verify the student implementation. Those library comparisons are not counted.

Random values and the fixed seed

```
srand(20261005);
arr[i] = rand() % 1000;
```

The generated values range from 0 to 999, so duplicates can occur. A fixed seed starts the same pseudorandom sequence on the same C-library implementation. All four programs make the same number of random calls, so they receive matching input arrays. Different C libraries can produce different sequences for the same seed.

Calculating the mean

If ten recorded counters total 2,490, their average is 249.0. Division by 10.0 retains a fractional result. `static_cast<double>` explicitly converts the integer total to floating point. The counter and average are experimental quantities; neither is a time measurement.

Three saved outputs per comparison program

Output	What it contains
Console and *_output.txt	Two columns: size and average counter.
*_comparison_trials.csv	n, trial number and the individual counter.
*_comparisons.csv	n, number of trials, mean counter, $n \log_2(n)$, $n \log_{10}(n)$.

The CSV reference curves are calculated with $\text{size} * \log_2(\text{size})$ and $\text{size} * \log_{10}(\text{size})$. Plot scripts use these saved values rather than timing or sorting the data in Python.

3. Program 1: Insertion sort

File: src/insertion_sort.cpp

Aim: Sort by inserting the next element into the already sorted prefix and evaluate the supplied shift-only counter.

The idea and invariant

At the beginning of pass i , indices 0 through $i - 1$ are sorted. Save $\text{arr}[i]$ as key, shift larger values one place right, then place key into the gap. After the pass, indices 0 through i are sorted. Repeating this extends the sorted prefix until it includes the complete array.

```
// Aim: Implement insertion sort and measure the supplied shift counter.
// Insert each key into its sorted prefix. Best  $O(n)$ ; average/worst  $O(n^2)$ .

// Match the supplied lab example: count successful shifts only.
void insertionSort(int arr[], int n, int &comparisons) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            comparisons++;
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Code walkthrough

Statement	Reason
$i = 1$	The first element alone is already sorted.
$\text{key} = \text{arr}[i]$	Save the element before shifting overwrites its original location.
$j = i - 1$	Start with the rightmost value in the sorted prefix.
$j \geq 0 \ \&\& \ \text{arr}[j] > \text{key}$	Continue only while an existing value must move right.
$\text{comparisons}++$	Count a successful shift under the supplied lab convention.
$\text{arr}[j + 1] = \text{arr}[j]$	Move a larger value into the next position.
$j--$	Inspect the next value to the left.
$\text{arr}[j + 1] = \text{key}$	Insert the saved value where the shifting stopped.

Worked dry run: [5, 3, 4, 1]

Pass / key	Changes	After insertion	Count added
i=1, key=3	Shift 5; insert 3 at index 0.	[3, 5, 4, 1]	1
i=2, key=4	Shift 5; stop at 3; insert 4.	[3, 4, 5, 1]	1
i=3, key=1	Shift 5, 4 and 3; insert 1.	[1, 3, 4, 5]	3

The total recorded counter is 5. For the shorter example [5, 3, 4], it is 2. Although $3 > 4$ is evaluated and is false, the loop body does not execute for that check, so the counter does not increase.

Exactly what is counted

This counter equals the number of **inversions** in the original input: pairs of indices $i < j$ for which $\text{arr}[i] > \text{arr}[j]$. Each shift removes one such out-of-order pair. A sorted input records zero, even though the program still checks values and performs loop work. Do not describe this counter as all executed key comparisons.

Complexity, space and stability

Case	Work / recorded counter	Time
Best: ascending	No shifts; counter = 0.	$O(n)$
Average: random	About quadratic shift work; roughly $n^2/4$ for distinct random values.	$O(n^2)$
Worst: reverse, distinct	$1 + 2 + \dots + (n-1)$ shifts = $n(n-1)/2$.	$O(n^2)$

The lab random arrays can contain duplicates, so $n^2/4$ is only a rough guide, not an exact prediction. Auxiliary space is $O(1)$. The algorithm is stable: equal values are not shifted past one another because the condition uses $>$, not $>=$.

Questions you should be able to answer

Why save key?

Without it, shifting could overwrite the value that still needs to be inserted.

Why insert at $j + 1$?

On exit, j either identifies a value not larger than key or is -1. The gap is immediately to its right.

Is a zero best-case counter the same as zero work?

No. The chosen counter excludes failed comparisons; the best-case runtime is linear.

4. Program 2: Merge sort

File: src/merge_sort.cpp

Aim: Apply divide and conquer, and count comparisons between current values of the two sorted halves.

Three responsibilities

mergeSort starts the full range. mergeRange recursively divides and sorts it. mergeParts combines two adjacent sorted halves into a temporary array and copies the result back. The endpoints left and right are inclusive.

```
// Aim: Implement recursive merge sort and count value comparisons.
// Divide into two halves, sort each half, and merge. Time O(n log n).

void mergeParts(int arr[], int left, int middle, int right, int &comparisons) {
    int temp[1000];
    int i = left, j = middle + 1, k = left;
    while (i <= middle && j <= right) {
        comparisons++; // One comparison between values in the two halves.
        if (arr[i] <= arr[j]) temp[k++] = arr[i++];
        else temp[k++] = arr[j++];
    }
    while (i <= middle) temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];
    for (int index = left; index <= right; index++) arr[index] = temp[index];
}

void mergeRange(int arr[], int left, int right, int &comparisons) {
    if (left >= right) return;
    int middle = left + (right - left) / 2;
    mergeRange(arr, left, middle, comparisons);
    mergeRange(arr, middle + 1, right, comparisons);
    mergeParts(arr, left, middle, right, comparisons);
}

void mergeSort(int arr[], int n, int &comparisons) {
    mergeRange(arr, 0, n - 1, comparisons);
}
```

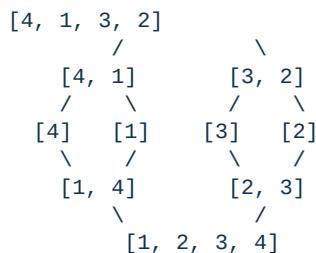
Understanding the recursive calls

The condition `left >= right` handles a range of zero or one element. Otherwise, `middle = left + (right - left) / 2` divides the range. The calls cover `[left, middle]` and `[middle + 1, right]`, so no element is omitted or included twice. The merge happens only after both recursive calls finish.

Understanding the merge

`i` points into the left half, `j` into the right half, and `k` into the temporary array. At each comparison the smaller current value is copied. Post-increment expressions such as `temp[k++] = arr[i++]` mean copy using the current indices, then increase both indices. Once a half is exhausted, the remaining half is already ordered, so its leftover values are simply copied.

Worked split and merge



Merge	Value comparisons	Count
[4] with [1]	4 <= 1 is false; take 1, then copy 4.	1
[3] with [2]	3 <= 2 is false; take 2, then copy 3.	1
[1,4] with [2,3]	1 <= 2, 4 <= 2, 4 <= 3; then copy 4.	3

Total counter: 5. False comparisons count because the increment happens before the if statement. Copying leftovers does not count as a value comparison.

Why it is correct and how it grows

The merge invariant is that the part already written to temp contains the smallest values encountered so far in sorted order. Since both halves are sorted, selecting the smaller current value is safe. The recursion ensures that this property applies at every merge level.

There are about $\log_2(n)$ levels, and each level merges a total of $O(n)$ elements. The recurrence $T(n) = 2T(n/2) + O(n)$ therefore gives $O(n \log n)$ time in all cases. Textbook temporary storage is $O(n)$, plus $O(\log n)$ recursion depth. This lab reserves `temp[1000]`, so its maximum supported input size is 1,000.

Merge sort is stable here: when values are equal, $\leq$ selects the value from the left half first. This implementation is not in-place because it uses a separate temporary array.

Questions you should be able to answer

Why is `middle + 1` the start of the right half?

`middle` already belongs to the left half; this makes the halves disjoint.

Why is the worst case still $O(n \log n)$?

The splits remain balanced and every merge level performs linear work, regardless of input order.

Why not compare leftovers?

No opposite half remains; the leftovers are already sorted.

5. Program 3: Quick sort

File: src/quick_sort.cpp

Aim: Partition around the last element and recursively sort both sides. This implementation uses Lomuto partitioning.

```
// Aim: Implement recursive quick sort and count pivot comparisons.
// Lomuto partition with last-element pivot. Average  $O(n \log n)$ , worst  $O(n^2)$ .

int partitionValues(int arr[], int low, int high, int &comparisons) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        comparisons++; // Compare each value with the pivot.
        if (arr[j] <= pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickRange(int arr[], int low, int high, int &comparisons) {
    if (low >= high) return;
    int pivotIndex = partitionValues(arr, low, high, comparisons);
    quickRange(arr, low, pivotIndex - 1, comparisons);
    quickRange(arr, pivotIndex + 1, high, comparisons);
}

void quickSort(int arr[], int n, int &comparisons) {
    quickRange(arr, 0, n - 1, comparisons);
}
```

The partition variables

pivot saves arr[high]. The index i marks the end of the portion known to contain values less than or equal to the pivot; initially that portion is empty, so i = low - 1. The index j scans low through high - 1. A qualifying value is swapped into the next position of the left portion.

After the scan, swap(arr[i + 1], arr[high]) places the pivot between the two portions. The returned index is its final sorted position. Values on the left are <= pivot, and values on the right are > pivot. The pivot is excluded from both recursive calls.

Worked partition: [4, 1, 3, 2]

Step	Test / action	Array and i
Start	pivot=2, low=0, high=3.	[4,1,3,2], i=-1
j=0	4 <= 2 is false; no swap.	[4,1,3,2], i=-1
j=1	1 <= 2 is true; i=0; swap arr[0], arr[1].	[1,4,3,2], i=0
j=2	3 <= 2 is false; no swap.	[1,4,3,2], i=0
Place pivot	Swap arr[1] with arr[3]; return 1.	[1,2,3,4]

This first partition performs three pivot comparisons. The right range [3,4] then makes one more comparison. The complete sort records four comparisons. A partition is not guaranteed to sort both sides immediately; this small example happens to leave them ordered.

Counter, recursion and complexity

The counter increases once for every `arr[j] <= pivot` evaluation, whether true or false. It does not count swaps, index tests or validation. A partition of m elements records $m - 1$ pivot comparisons.

Balanced partitions give $T(n) = 2T(n/2) + O(n)$, hence $O(n \log n)$. One-sided partitions give $T(n) = T(n - 1) + O(n)$, hence $O(n^2)$. With the last-element pivot, ascending, descending and all-equal inputs can produce the worst case. Average-case $O(n \log n)$ depends on the input distribution or randomization assumptions; it is not guaranteed for every input.

This version uses ordinary recursion. Stack space is $O(\log n)$ for balanced partitions and $O(n)$ in the worst case. Apart from that stack, partitioning uses constant extra storage. The algorithm is unstable because swaps may change the order of equal records.

A useful exact count

On ascending input of n distinct values, the pivot is always the largest. The partitions record $(n - 1) + (n - 2) + \dots + 1 = n(n - 1)/2$ comparisons. The correctness tests verify this count for small inputs.

Questions you should be able to answer

What is the difference between partitioning and sorting?

Partitioning puts one pivot in its final position and separates the sides; the sides still require recursive sorting.

Why return $i + 1$?

That is where the pivot was placed after the scan.

Why does the pivot choice matter?

It controls the balance of the recursive subproblems and therefore the runtime and stack depth.

6. Program 4: Heap sort

File: src/heap_sort.cpp

Aim: Build a max heap, then repeatedly put its maximum at the end of the unsorted range.

A heap is an array representing a complete binary tree

For a node at index $root$, its left child is $2 * root + 1$ and its right child is $2 * root + 2$. In a max heap every parent is at least as large as its children. The root therefore holds the maximum. A heap is not a fully sorted array: sibling order is unrestricted.

```
// Aim: Implement heap sort and count value comparisons.
// Build a max heap, then repeatedly place its root at the end. Time  $O(n \log n)$ .
```

```
void heapify(int arr[], int n, int root, int &comparisons) {
    int largest = root;
    int left = 2 * root + 1;
    int right = 2 * root + 2;
    if (left < n) {
        comparisons++;
        if (arr[left] > arr[largest]) largest = left;
    }
    if (right < n) {
        comparisons++;
        if (arr[right] > arr[largest]) largest = right;
    }
    if (largest != root) {
        swap(arr[root], arr[largest]);
        heapify(arr, n, largest, comparisons);
    }
}

void heapSort(int arr[], int n, int &comparisons) {
    for (int i = n / 2 - 1; i >= 0; i--) heapify(arr, n, i, comparisons);
    for (int end = n - 1; end > 0; end--) {
        swap(arr[0], arr[end]);
        heapify(arr, end, 0, comparisons);
    }
}
```

How heapify repairs one subtree

largest starts at root. If a left child exists, compare it with the current largest. Do the same for the right child. If a child wins, swap that child with the root and recursively repair the affected child subtree. heapify assumes the child subtrees are already heaps; it repairs the possible violation at their parent.

Why construction starts at $n / 2 - 1$

Indices $n / 2$ through $n - 1$ are leaves, which already satisfy the heap property. Processing internal nodes backward ensures each node's child subtrees have been repaired before that node is processed.

Worked dry run: [4, 1, 3, 2]

Stage	Active heap / action	Array	Count added
Build: root=1	Compare child 2 with parent 1; swap.	[4,2,3,1]	1
Build: root=0	Compare 2 and 3 with current maximum 4.	[4,2,3,1]	2
Extract 4	Swap root with index 3; heapify first 3 values.	[3,2,1,4]	2
Extract 3	Swap root with index 2; heapify first 2 values.	[2,1,3,4]	1
Extract 2	Swap root with index 1; heap size is 1.	[1,2,3,4]	0

Total recorded comparisons: 6. The right-child comparison is with `arr[largest]`, which may already refer to the left child, so heapify selects the maximum of all available candidates.

Complexity, space and stability

Bottom-up heap construction is $O(n)$: most nodes are near the leaves and require little downward movement. Each of the $n - 1$ extractions may perform $O(\log n)$ repair work, so the complete algorithm is $O(n \log n)$. It has no quadratic worst case like last-pivot quick sort.

The array is rearranged in-place. This implementation's recursive heapify adds $O(\log n)$ stack space; an iterative heapify can use $O(1)$ auxiliary space. Heap sort is unstable because distant swaps can reorder equal records.

Questions you should be able to answer**Why use a max heap for ascending output?**

Placing the current maximum at the right end builds the sorted suffix in ascending order.

Why pass end as the heap size?

The element at end has just been fixed in the sorted suffix, so it must no longer participate in heap comparisons.

Does checking `left < n` count?

No. It is an index-bound check, not a comparison of array values.

7. Program 5: Insertion sort timing

File: src/insertion_sort_timing.cpp
Aim: Measure the best, average and worst cases for exactly 15,000 distinct elements, in milliseconds.

Same algorithm, a different experiment

insertionSortTimed performs the same shifting and insertion operations as Program 1, but it has no comparison counter. The experiment fixes n at 15,000 and changes the input order. Keeping a counter in this timed task would add work that is not part of the sorting operations.

```
// Aim: Measure insertion sort for best, average and worst cases, n = 15000.
void insertionSortTimed(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

// caseNumber: 0 = ascending, 1 = random permutation, 2 = descending.
void prepareInsertionInput(int arr[], int n, int caseNumber) {
    for (int i = 0; i < n; i++) {
        if (caseNumber == 2) arr[i] = n - i;
        else arr[i] = i + 1;
    }
    if (caseNumber == 1) {
        // Fisher-Yates shuffle gives distinct elements in a random order.
        for (int i = n - 1; i > 0; i--) {
            int j = rand() % (i + 1);
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
}
```

What prepareInsertionInput does

caseNumber	Initial values	Reason
0: best	1, 2, ..., 15000	Already sorted: no shifts.
1: average	Start with 1..15000, then shuffle.	Random order: represents an average-case experiment.
2: worst	15000, 14999, ..., 1	Each new key must move to the beginning.

The descending loop in the shuffle chooses an index j in 0..i and exchanges arr[i] with arr[j]. This is the Fisher-Yates structure. It preserves all values exactly once. rand() modulo a range is suitable for this classroom experiment but can introduce small distribution bias; it is not a claim of perfectly uniform sampling.

The timed part of main()

```
prepareInsertionInput(arr, n, caseNumber);
auto start = chrono::steady_clock::now();
insertionSortTimed(arr, n);
auto end = chrono::steady_clock::now();
double milliseconds =
    chrono::duration<double, milli>(end - start).count();
```

The input is prepared before start. Correctness checking and CSV writing occur after end. `steady_clock` is monotonic, making it suitable for elapsed durations. `duration<double, milli>` expresses the difference in fractional milliseconds rather than truncating it to a whole integer.

Five trials per case

`main()` runs one warmup before measuring each case, then records five trials. A fresh array is prepared each time. The random case receives a new permutation per trial. The mean is `totalMilliseconds / trials`. Validation checks every result against 1, 2, ..., 15000; it is outside the timed interval.

Case	Elements	Trials	Recorded mean (ms)
Best	15000	5	0.008208
Average	15000	5	32.048183
Worst	15000	5	56.559575

These means were measured on macOS 26.2, ARM64, using Apple Clang 17.0.0 and `-O2`. The best case is $O(n)$; random and reverse cases are $O(n^2)$. The shift counts for 15,000 distinct elements would be 0 in the best case and 112,492,500 in the worst case, but this timed version does not record them.

Questions you should be able to answer

Why not reuse the sorted result in the next trial?

That would turn later trials into the best case, even when measuring random or reverse input.

Are milliseconds the same as time complexity?

No. Milliseconds are one measured duration; complexity describes growth as input size changes.

Why exclude input generation?

The requested quantity is sorting execution time, so generation and output should not inflate it.

8. Program 6a: Radix sort

File: src/radix_sort.cpp

Aim: Sort nonnegative integers by their decimal digits, using stable counting sort in each pass.

Least significant digit first

LSD radix sort processes units, tens, hundreds and so on. A later digit pass must preserve the ordering established by earlier passes when current digits are equal. This is why every digit pass must be stable.

```
// Aim: Sort nonnegative integers using LSD radix sort.
// Method: Apply stable counting sort to units, tens, hundreds, ... digits.
void sortByDigit(int arr[], int n, long long exponent) {
    int count[10] = {0};
    int *output = new int[n];
    for (int i = 0; i < n; i++) count[(arr[i] / exponent) % 10]++;
    for (int digit = 1; digit < 10; digit++) count[digit] += count[digit - 1];
    // Traversing backwards preserves the order of equal digits (stability).
    for (int i = n - 1; i >= 0; i--) {
        int digit = static_cast<int>((arr[i] / exponent) % 10);
        output[count[digit] - 1] = arr[i];
        count[digit]--;
    }
    for (int i = 0; i < n; i++) arr[i] = output[i];
    delete[] output;
}

bool radixSort(int arr[], int n) {
    if (n < 0) return false;
    if (n == 0) return true;
    int maximum = arr[0];
    for (int i = 0; i < n; i++) {
        if (arr[i] < 0) return false;
        if (arr[i] > maximum) maximum = arr[i];
    }
    // long long prevents overflow when exponent passes INT_MAX's last digit.
    for (long long exponent = 1; maximum / exponent > 0; exponent *= 10)
        sortByDigit(arr, n, exponent);
    return true;
}
```

Reading the digit expression

```
int digit = (arr[i] / exponent) % 10;
```

For 802: exponent = 1 gives $802 \% 10 = 2$; exponent = 10 gives $80 \% 10 = 0$; exponent = 100 gives $8 \% 10 = 8$. Integer division discards the lower digits, and modulo 10 selects the next digit.

How sortByDigit works

- **Frequency:** count[digit] stores how many values have that digit.
- **Prefix sums:** accumulating counts tells us how many values have digits up to each digit.
- **Placement:** scan backward, place a value at count[digit] - 1, then decrement that count.
- **Copy back:** write output into arr so the next pass sees the previous pass's order.
- **Release:** delete[] output frees the dynamically allocated temporary array.

Worked dry run: [170, 45, 75, 90]

Pass	Digit grouping	Array after stable pass
Units, exp=1	Digits: 0,5,5,0.	[170,90,45,75]
Tens, exp=10	Digits of current order: 7,9,4,7.	[45,170,75,90]
Hundreds, exp=100	Digits: 0,1,0,0.	[45,75,90,170]

When sorting the tens digits, 170 and 75 both have digit 7. Keeping their previous order preserves the information established by the units pass. The loop stops when maximum / exponent becomes zero.

Limits, complexity and demonstration output

This version rejects negative values. Empty arrays are accepted. If every value is zero, no digit pass is needed. The exponent is long long so multiplying it by 10 after the final int digit does not overflow an int. For d decimal digits, time is $O(d(n + 10))$ and auxiliary space is $O(n + 10)$. The algorithm is stable and not in-place.

The demo uses 25 integers and 1,000 measured runs. Each call includes its own digit-array allocation and input validation; the input copy and final reference check are outside the timed interval. The recorded mean is **0.000768 ms**.

Questions you should be able to answer

Why can we not use an unstable digit sort?

It could reorder equal current digits and destroy the lower-digit ordering built by earlier passes.

What does LSD mean?

Least significant digit: the units digit is processed first.

Is radix sort based on pairwise key comparisons?

Its main ordering work uses digit indexing and counting, so it is a noncomparison sorting method.

9. Program 6b: Bucket sort

File: src/bucket_sort.cpp

Aim: Sort finite fractional values in $[0, 1)$, using n bucket lists and insertion sort within each list.

```
// Aim: Sort fractions in  $[0, 1)$  using textbook bucket sort.
void insertionSortBucket(vector<double> &bucket) {
    for (int i = 1; i < static_cast<int>(bucket.size()); i++) {
        double key = bucket[i];
        int j = i - 1;
        while (j >= 0 && bucket[j] > key) {
            bucket[j + 1] = bucket[j];
            j--;
        }
        bucket[j + 1] = key;
    }
}

bool bucketSort(double arr[], int n) {
    if (n < 0) return false;
    if (n == 0) return true;
    for (int i = 0; i < n; i++)
        if (!isfinite(arr[i]) || arr[i] < 0.0 || arr[i] >= 1.0) return false;

    // Create n bucket lists; vector stores a variable number of values per list.
    vector<double> *buckets = new vector<double>[n];
    for (int i = 0; i < n; i++) {
        int index = static_cast<int>(n * arr[i]);
        if (index >= n) index = n - 1;
        buckets[index].push_back(arr[i]);
    }
    int next = 0;
    for (int i = 0; i < n; i++) {
        insertionSortBucket(buckets[i]);
        for (int j = 0; j < static_cast<int>(buckets[i].size()); j++)
            arr[next++] = buckets[i][j];
    }
    delete[] buckets;
    return true;
}
```

Distribution, local sorting and concatenation

n buckets divide $[0, 1)$ into n equal intervals. For nonnegative values, `static_cast<int>(n * arr[i])` truncates to the floor and chooses the bucket. A vector stores a variable number of values in each bucket. `push_back` preserves their input order. After insertion sorting each bucket, the values are copied back in increasing bucket-index order.

The upper guard index `>= n` clamps a possible floating-point rounding edge to the last bucket. Domain validation happens first: negative numbers, 1.0, infinities and NaN are rejected. Empty arrays return successfully without allocating buckets.

Worked dry run: [0.42, 0.12, 0.35, 0.09]

Input value	$\text{floor}(4 * \text{value})$	Bucket
0.42	1	B1: [0.42]
0.12	0	B0: [0.12]
0.35	1	B1: [0.42,0.35]
0.09	0	B0: [0.12,0.09]

Before local sorting:

B0 = [0.12, 0.09] B1 = [0.42, 0.35] B2 = [] B3 = []

After local insertion sorts:

B0 = [0.09, 0.12] B1 = [0.35, 0.42]

Concatenate: [0.09, 0.12, 0.35, 0.42]

Why concatenation gives a sorted array

Every value assigned to an earlier bucket is less than every value assigned to a later bucket, apart from boundary rounding handled by the last-bucket guard. Once each bucket is internally sorted, concatenating them preserves the complete ascending order. Equal values go to the same bucket, so stable distribution and stable insertion sorting preserve their order.

Complexity depends on the distribution

If bucket i contains n_i values, insertion sorting contributes $O(n_i^2)$ in the worst case for that bucket. Total work is $O(n + \sum(n_i^2))$, including distribution and concatenation. With n buckets and approximately uniform inputs, expected work is $O(n)$. If all values concentrate in one bucket in an unfavorable order, time can be $O(n^2)$. Auxiliary space is $O(n)$ for the bucket objects and their stored values.

The demo uses the same 25 integer values as radix and counting sort, divided by 1,000. Its recorded mean over 1,000 runs is **0.000821 ms**. Allocation of the bucket array and vector storage is included in the sort duration.

Questions you should be able to answer**Why is 1.0 excluded?**

$\text{floor}(n * 1.0)$ would be n , outside valid bucket indices $0..n-1$.

Why use insertion sort inside a bucket?

Under a favorable distribution each bucket is small; insertion sort is simple and stable.

Is linear time guaranteed?

No. Expected linear time depends on the input distribution; the worst case is quadratic.

10. Program 6c: Counting sort

File: src/counting_sort.cpp

Aim: Sort nonnegative integers from a reasonably small range using frequencies, cumulative counts and stable output placement.

```
// Aim: Sort nonnegative integers using stable counting sort.
// Method: Count frequencies, form prefix sums, and place elements backwards.
bool countingSort(int arr[], int n) {
    if (n < 0) return false;
    if (n == 0) return true;
    int maximum = arr[0];
    for (int i = 0; i < n; i++) {
        // Keep the frequency table within a reasonable memory limit.
        if (arr[i] < 0 || arr[i] > 1000000) return false;
        if (arr[i] > maximum) maximum = arr[i];
    }
    int *count = new int[maximum + 1](); // Initialize every frequency to zero.
    int *output = new int[n];
    for (int i = 0; i < n; i++) count[arr[i]]++;
    for (int value = 1; value <= maximum; value++) count[value] += count[value - 1];
    for (int i = n - 1; i >= 0; i--) {
        output[count[arr[i]] - 1] = arr[i];
        count[arr[i]]--;
    }
    for (int i = 0; i < n; i++) arr[i] = output[i];
    delete[] count;
    delete[] output;
    return true;
}
```

Maximum and memory allocation

The program checks the domain and finds maximum. count needs indices 0 through maximum, so it has maximum + 1 entries. The parentheses in new int[maximum + 1]() zero-initialize every entry. output has n entries. After copying back, both allocated arrays are released with delete[].

What cumulative counts mean

After the frequency loop, count[v] is the number of occurrences of value v. After the prefix-sum loop, count[v] is the number of input values less than or equal to v. Therefore count[v] - 1 is the last available zero-based position for v. Decrementing the count makes the previous position available for the next equal occurrence.

Worked dry run: [2, 1, 2, 0]

Stage	State
Frequency count	count[0..2] = [1,1,2]
Prefix sums	count[0..2] = [1,2,4]
i=3, value=0	Place 0 at output[0]; count becomes [0,2,4].
i=2, value=2	Place 2 at output[3]; count becomes [0,2,3].
i=1, value=1	Place 1 at output[1]; count becomes [0,1,3].
i=0, value=2	Place 2 at output[2]; count becomes [0,1,2].
Final output	[0,1,2,2]

Why scanning backwards makes it stable

Imagine the two 2s are labeled 2A and 2B in their original order. The backward scan places 2B into the later position first, then 2A into the earlier position. Their original relative order is preserved. This stable placement strategy is also used inside radix sort.

Complexity, range and actual demonstration

Let $k = \text{maximum} + 1$. Counting frequencies and placing output take $O(n)$; initializing and accumulating the count array take $O(k)$. Total time and auxiliary space are $O(n + k)$. Calling the algorithm linear in n alone assumes k is also bounded suitably. A huge maximum with only a few elements can waste memory and time.

This implementation accepts values from 0 through 1,000,000 and returns false for unsupported values. Its classroom demo uses 25 elements with maximum 999. The recorded mean over 1,000 runs is **0.000668 ms**. Allocating and zero-initializing the count array is included in the duration.

Questions you should be able to answer

Why maximum + 1 entries?

Zero is a valid value; including maximum requires indices 0 through maximum.

Why subtract one from the cumulative count?

Counts describe a quantity, but C++ array positions start at zero.

When is counting sort a poor choice?

When the range is very large compared with n , since the frequency array dominates the work and memory.

11. Timing, results and graph interpretation

The common timing driver for the three 25-element tasks

Each demo creates input, copies it to expected, and library-sorts expected for validation. It performs one warmup. It then repeats the following measured trial 1,000 times, keeping the initial data identical:

```
for (int trial = 1; trial <= trials; trial++) {
    for (int i = 0; i < n; i++) arr[i] = input[i];
    auto start = chrono::steady_clock::now();
    bool success = radixSort(arr, n); // Or bucketSort / countingSort.
    auto end = chrono::steady_clock::now();
    double milliseconds =
        chrono::duration<double, milli>(end - start).count();
    // Validate success and exact output outside the interval.
    totalMilliseconds += milliseconds;
}
double averageMilliseconds = totalMilliseconds / trials;
```

The bool return tells main() whether the input domain was supported. The full program validates every measured output and saves each duration before reporting the mean. Input copying, reference sorting, printing and CSV output are excluded. Domain validation and allocations inside the algorithm are included.

The 25-element data and sorted result

Integer input:
 170 45 75 90 802 24 2 66 0 999 501 18 39 120 300
 5 75 42 610 7 88 456 321 11 200

Sorted integers:
 0 2 5 7 11 18 24 39 42 45 66 75 75 88 90
 120 170 200 300 321 456 501 610 802 999

Bucket sort uses these values divided by 1,000. Its expected result is the sorted integer sequence divided by 1,000. Notice the two occurrences of 75, or 0.075: correct sorting retains both.

Algorithm	Elements	Measured trials	Mean milliseconds
Radix Sort	25	1000	0.000768
Bucket Sort	25	1000	0.000821
Counting Sort	25	1000	0.000668

Interpreting small execution times

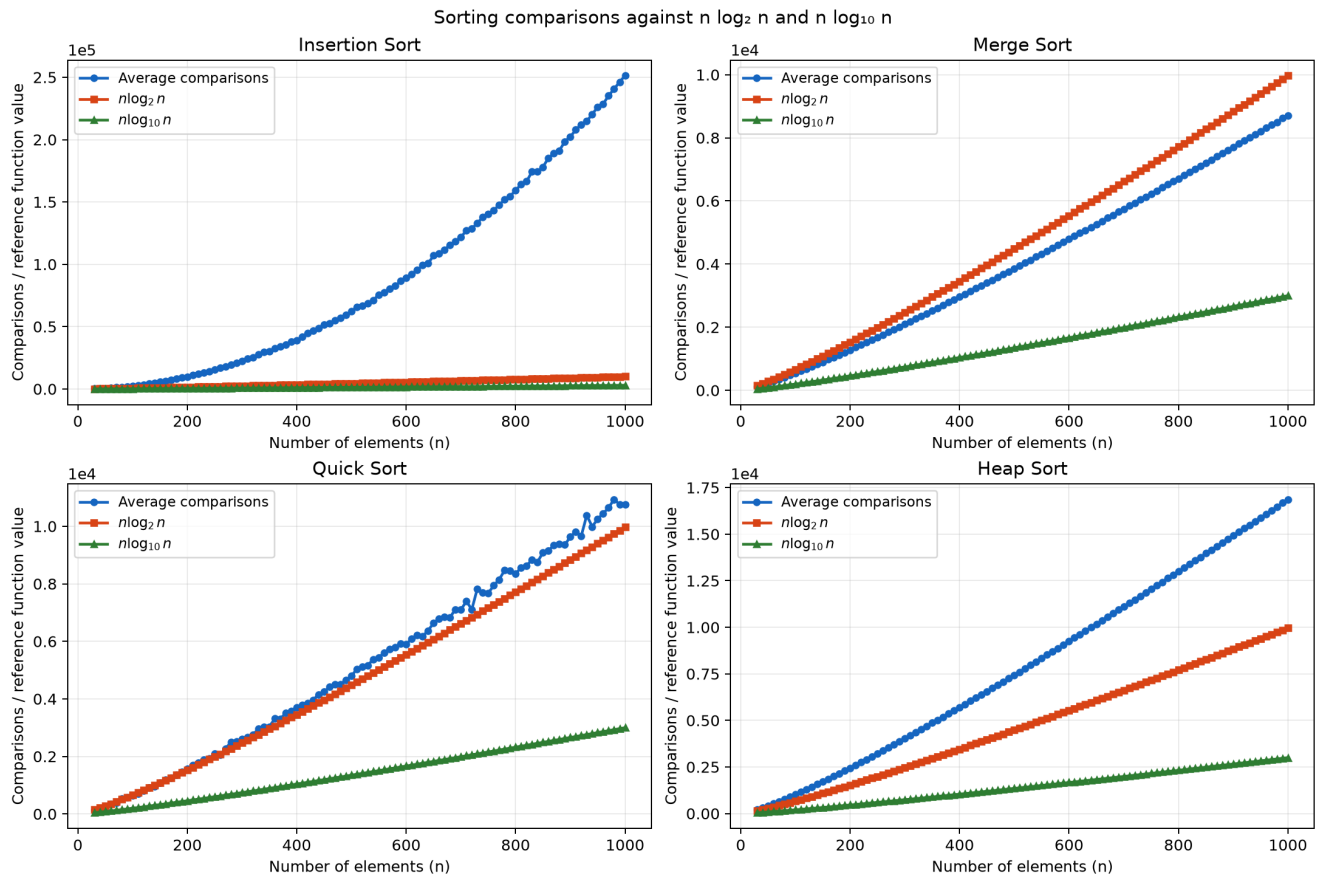
The durations for 25 elements are much smaller than one millisecond. Floating-point milliseconds preserve those fractional values; rounding to an integer would often show zero. Timer overhead and machine load still affect the measurements. Averaging reduces some noise but does not prove one algorithm is universally faster.

Representative comparison measurements

n	Insertion shifts	Merge	Quick	Heap
30	216.50	109.80	123.30	205.20
100	2,490.00	540.50	640.60	1,025.60
250	15,648.30	1,680.50	2,102.70	3,213.20
500	62,225.50	3,855.90	4,793.50	7,417.20
750	140,412.50	6,218.70	7,676.90	12,065.10
1000	251,191.50	8,709.10	10,764.70	16,858.80

The complete CSV data contains all 98 sizes. Insertion is deliberately labeled shifts here; the other counters include both true and false executed value comparisons. Compare growth shapes carefully rather than treating these as identical definitions of total work.

The four graph panels



Average counters against $n \log_2(n)$ and $n \log_{10}(n)$, ten trials per size. Each panel has its own vertical scale.

What the curves tell you

- **Insertion:** the recorded average shifts grow approximately quadratically on random input. This eventually exceeds either logarithmic reference curve.
- **Merge:** the balanced recursive structure gives $n \log n$ growth. The exact count can lie below $n \log_2(n)$ because leftover copies require no cross-half comparisons.
- **Quick:** its random-input average generally follows $n \log n$ growth, with trial-to-trial fluctuations from pivot balance.
- **Heap:** construction and repeated repairs give $n \log n$ growth; its constants differ from merge and quick.

The logarithm base changes only a constant: $\log_2(n) = \log_{10}(n) / \log_{10}(2)$. Therefore $n \log_2(n)$ is about 3.322 times $n \log_{10}(n)$. Both curves represent the same asymptotic growth class. Neither is an exact universal comparison formula or a millisecond curve.

For a plot, use n on the horizontal axis and the average counter or reference value on the vertical axis. A graph for one fixed size cannot establish a growth rate; the varied-size experiment is what makes the shape visible.

12. Revision sheet and running the programs

Algorithm	Best / average / worst time	Auxiliary space	Stable?
Insertion	$O(n)$ / $O(n^2)$ / $O(n^2)$	$O(1)$	Yes
Merge	$O(n \log n)$ in all cases	$O(n)$ temp + recursion; lab cap 1000	Yes
Quick	$O(n \log n)$ / $O(n \log n)$ / $O(n^2)$	$O(\log n)$ average stack; $O(n)$ worst	No
Heap	$O(n \log n)$ as the usual case bound	$O(\log n)$ recursive stack	No
Radix	$O(d(n + 10))$	$O(n + 10)$	Yes
Bucket	Expected $O(n)$ under uniform input; worst $O(n^2)$	$O(n)$	Yes, in this version
Counting	$O(n + k)$	$O(n + k)$	Yes

d is the number of decimal digits; k is maximum + 1. Stability means equal keys retain their relative input order. In-place array rearrangement and auxiliary recursion space are separate considerations.

Fixed-capacity implementation details must be distinguished from the general textbook space bound.

Compile one file

```
c++ -std=c++17 -O2 -Wall -Wextra -Wpedantic src/insertion_sort.cpp -o insertion_sort
./insertion_sort
```

Run from the repository root. Each .cpp has its own main() and compiles independently; do not link all eight together. C++17 is required by the result-directory support. The algorithm itself is expressed through ordinary arrays, loops and functions.

Compile, test and run the complete lab

```
make all
make test
make sanitize
make run
```

The tests check all permutations through seven elements for the comparison sorts, random duplicate-heavy inputs, boundaries, exact counter cases, insertion counters against inversion counts, invalid domains, and the construction of all three 15,000-element input orders. AddressSanitizer and UndefinedBehaviorSanitizer checks passed for this source version.

Generate report and graphs after a new experiment

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python scripts/summarize_results.py
.venv/bin/python scripts/plot_comparisons.py
```

On Windows use .venv\Scripts\python.exe. Running a C++ program overwrites its corresponding result files. Regenerate the report and graphs after changing experiment data. This PDF is a dated explanation of the source and measurements identified on its cover; it does not update automatically.

Common mistakes to avoid

- Not resetting the counter for each trial, or the total for each new size.
- Reusing a sorted array instead of generating or copying fresh input.
- Calling the insertion shift-only counter a count of all comparisons.
- Including printing, data generation or reference sorting inside the timer.
- Using integer division when converting timer units or calculating an average.
- Forgetting input-domain restrictions, auxiliary storage or recursion depth.

Appendix A. Complete C++ source listings

The listings below reproduce the eight files from source version 14b5fd3. Line numbers are added for study; long lines may wrap visually. Remove line numbers when copying code. All supporting logic, validation and result-file output is included.

insertion_sort.cpp

Source lines 1 through 38.

```

1  #include <iostream>
2  #include <cstdlib>
3  #include <cmath>
4  #include <fstream>
5  #include <iomanip>
6  #include <algorithm>
7  #include <filesystem>
8
9  using namespace std;
10
11 // Aim: Implement insertion sort and measure the supplied shift counter.
12 // Insert each key into its sorted prefix. Best O(n); average/worst O(n^2).
13
14 // Match the supplied lab example: count successful shifts only.
15 void insertionSort(int arr[], int n, int &comparisons) {
16     for (int i = 1; i < n; i++) {
17         int key = arr[i];
18         int j = i - 1;
19         while (j >= 0 && arr[j] > key) {
20             comparisons++;
21             arr[j + 1] = arr[j];
22             j--;
23         }
24         arr[j + 1] = key;
25     }
26 }
27
28 #ifndef SORTING_TEST
29 int main() {
30     // A fixed seed lets all four programs use the same random arrays.
31     // Use srand(time(0)) with <ctime> if new arrays on every run are wanted.
32     srand(20261005);
33     filesystem::create_directories("results");
34     ofstream summary("results/insertion_sort_comparisons.csv");
35     ofstream samples("results/insertion_sort_comparison_trials.csv");
36     ofstream output("results/insertion_sort_output.txt");
37     if (!summary || !samples || !output) {
38         cerr << "Cannot open result files" << endl;

```

insertion_sort.cpp - continued

Source lines 39 through 75.

```

39         return 1;
40     }
41     summary << fixed << setprecision(6);
42     output << fixed << setprecision(2);
43     cout << fixed << setprecision(2);
44     summary << "n, trials, average_comparisons, n_log2_n, n_log10_n\n";
45     samples << "n, trial, comparisons\n";
46
47     for (int size = 30; size <= 1000; size += 10) {
48         int totalComparisons = 0;
49         for (int instance = 0; instance < 10; instance++) {
50             // Standard C++ fixed-capacity arrays; only the first size values are used.
51             int arr[1000], expected[1000];
52             for (int i = 0; i < size; i++) {
53                 arr[i] = rand() % 1000;
54                 expected[i] = arr[i];
55             }
56             // std::sort is only a reference to check the lab algorithm's output.
57             sort(expected, expected + size);
58             int comparisons = 0;
59             insertionSort(arr, size, comparisons);
60             if (!equal(arr, arr + size, expected)) {
61                 cerr << "Incorrect sorted output" << endl;
62                 return 1;
63             }
64             totalComparisons += comparisons;
65             samples << size << ',' << instance + 1 << ',' << comparisons << '\n';
66         }
67         double averageComparisons = static_cast<double>(totalComparisons) / 10.0;
68         cout << size << " " << averageComparisons << endl;
69         output << size << " " << averageComparisons << '\n';
70         summary << size << ",10," << averageComparisons << ','
71             << size * log2(size) << ',' << size * log10(size) << '\n';
72     }
73     return 0;
74 }
75 #endif

```

merge_sort.cpp

Source lines 1 through 43.

```

1  #include <iostream>
2  #include <cstdlib>
3  #include <cmath>
4  #include <fstream>
5  #include <iomanip>
6  #include <algorithm>
7  #include <filesystem>
8
9  using namespace std;
10
11 // Aim: Implement recursive merge sort and count value comparisons.
12 // Divide into two halves, sort each half, and merge. Time O(n log n).
13
14 void mergeParts(int arr[], int left, int middle, int right, int &comparisons) {
15     int temp[1000];
16     int i = left, j = middle + 1, k = left;
17     while (i <= middle && j <= right) {
18         comparisons++; // One comparison between values in the two halves.
19         if (arr[i] <= arr[j]) temp[k++] = arr[i++];
20         else temp[k++] = arr[j++];
21     }
22     while (i <= middle) temp[k++] = arr[i++];
23     while (j <= right) temp[k++] = arr[j++];
24     for (int index = left; index <= right; index++) arr[index] = temp[index];
25 }
26
27 void mergeRange(int arr[], int left, int right, int &comparisons) {
28     if (left >= right) return;
29     int middle = left + (right - left) / 2;
30     mergeRange(arr, left, middle, comparisons);
31     mergeRange(arr, middle + 1, right, comparisons);
32     mergeParts(arr, left, middle, right, comparisons);
33 }
34
35 void mergeSort(int arr[], int n, int &comparisons) {
36     mergeRange(arr, 0, n - 1, comparisons);
37 }
38
39 #ifndef SORTING_TEST
40 int main() {
41     // A fixed seed lets all four programs use the same random arrays.
42     // Use srand(time(0)) with <ctime> if new arrays on every run are wanted.
43     srand(20261005);

```

merge_sort.cpp - continued

Source lines 44 through 86.

```

44     filesystem::create_directories("results");
45     ofstream summary("results/merge_sort_comparisons.csv");
46     ofstream samples("results/merge_sort_comparison_trials.csv");
47     ofstream output("results/merge_sort_output.txt");
48     if (!summary || !samples || !output) {
49         cerr << "Cannot open result files" << endl;
50         return 1;
51     }
52     summary << fixed << setprecision(6);
53     output << fixed << setprecision(2);
54     cout << fixed << setprecision(2);
55     summary << "n, trials, average_comparisons, n_log2_n, n_log10_n\n";
56     samples << "n, trial, comparisons\n";
57
58     for (int size = 30; size <= 1000; size += 10) {
59         int totalComparisons = 0;
60         for (int instance = 0; instance < 10; instance++) {
61             // Standard C++ fixed-capacity arrays; only the first size values are used.
62             int arr[1000], expected[1000];
63             for (int i = 0; i < size; i++) {
64                 arr[i] = rand() % 1000;
65                 expected[i] = arr[i];
66             }
67             // std::sort is only a reference to check the lab algorithm's output.
68             sort(expected, expected + size);
69             int comparisons = 0;
70             mergeSort(arr, size, comparisons);
71             if (!equal(arr, arr + size, expected)) {
72                 cerr << "Incorrect sorted output" << endl;
73                 return 1;
74             }
75             totalComparisons += comparisons;
76             samples << size << ',' << instance + 1 << ',' << comparisons << '\n';
77         }
78         double averageComparisons = static_cast<double>(totalComparisons) / 10.0;
79         cout << size << " " << averageComparisons << endl;
80         output << size << " " << averageComparisons << '\n';
81         summary << size << ",10," << averageComparisons << ','
82             << size * log2(size) << ',' << size * log10(size) << '\n';
83     }
84     return 0;
85 }
86 #endif

```

quick_sort.cpp

Source lines 1 through 43.

```
1  #include <iostream>
2  #include <cstdlib>
3  #include <cmath>
4  #include <fstream>
5  #include <iomanip>
6  #include <algorithm>
7  #include <filesystem>
8
9  using namespace std;
10
11 // Aim: Implement recursive quick sort and count pivot comparisons.
12 // Lomuto partition with last-element pivot. Average  $O(n \log n)$ , worst  $O(n^2)$ .
13
14 int partitionValues(int arr[], int low, int high, int &comparisons) {
15     int pivot = arr[high];
16     int i = low - 1;
17     for (int j = low; j < high; j++) {
18         comparisons++; // Compare each value with the pivot.
19         if (arr[j] <= pivot) {
20             i++;
21             swap(arr[i], arr[j]);
22         }
23     }
24     swap(arr[i + 1], arr[high]);
25     return i + 1;
26 }
27
28 void quickRange(int arr[], int low, int high, int &comparisons) {
29     if (low >= high) return;
30     int pivotIndex = partitionValues(arr, low, high, comparisons);
31     quickRange(arr, low, pivotIndex - 1, comparisons);
32     quickRange(arr, pivotIndex + 1, high, comparisons);
33 }
34
35 void quickSort(int arr[], int n, int &comparisons) {
36     quickRange(arr, 0, n - 1, comparisons);
37 }
38
39 #ifndef SORTING_TEST
40 int main() {
41     // A fixed seed lets all four programs use the same random arrays.
42     // Use srand(time(0)) with <ctime> if new arrays on every run are wanted.
43     srand(20261005);
```

quick_sort.cpp - continued

Source lines 44 through 86.

```

44     filesystem::create_directories("results");
45     ofstream summary("results/quick_sort_comparisons.csv");
46     ofstream samples("results/quick_sort_comparison_trials.csv");
47     ofstream output("results/quick_sort_output.txt");
48     if (!summary || !samples || !output) {
49         cerr << "Cannot open result files" << endl;
50         return 1;
51     }
52     summary << fixed << setprecision(6);
53     output << fixed << setprecision(2);
54     cout << fixed << setprecision(2);
55     summary << "n, trials, average_comparisons, n_log2_n, n_log10_n\n";
56     samples << "n, trial, comparisons\n";
57
58     for (int size = 30; size <= 1000; size += 10) {
59         int totalComparisons = 0;
60         for (int instance = 0; instance < 10; instance++) {
61             // Standard C++ fixed-capacity arrays; only the first size values are used.
62             int arr[1000], expected[1000];
63             for (int i = 0; i < size; i++) {
64                 arr[i] = rand() % 1000;
65                 expected[i] = arr[i];
66             }
67             // std::sort is only a reference to check the lab algorithm's output.
68             sort(expected, expected + size);
69             int comparisons = 0;
70             quickSort(arr, size, comparisons);
71             if (!equal(arr, arr + size, expected)) {
72                 cerr << "Incorrect sorted output" << endl;
73                 return 1;
74             }
75             totalComparisons += comparisons;
76             samples << size << ',' << instance + 1 << ',' << comparisons << '\n';
77         }
78         double averageComparisons = static_cast<double>(totalComparisons) / 10.0;
79         cout << size << " " << averageComparisons << endl;
80         output << size << " " << averageComparisons << '\n';
81         summary << size << ",10," << averageComparisons << ','
82             << size * log2(size) << ',' << size * log10(size) << '\n';
83     }
84     return 0;
85 }
86 #endif

```

heap_sort.cpp

Source lines 1 through 44.

```
1  #include <iostream>
2  #include <cstdlib>
3  #include <cmath>
4  #include <fstream>
5  #include <iomanip>
6  #include <algorithm>
7  #include <filesystem>
8
9  using namespace std;
10
11 // Aim: Implement heap sort and count value comparisons.
12 // Build a max heap, then repeatedly place its root at the end. Time  $O(n \log n)$ .
13
14 void heapify(int arr[], int n, int root, int &comparisons) {
15     int largest = root;
16     int left = 2 * root + 1;
17     int right = 2 * root + 2;
18     if (left < n) {
19         comparisons++;
20         if (arr[left] > arr[largest]) largest = left;
21     }
22     if (right < n) {
23         comparisons++;
24         if (arr[right] > arr[largest]) largest = right;
25     }
26     if (largest != root) {
27         swap(arr[root], arr[largest]);
28         heapify(arr, n, largest, comparisons);
29     }
30 }
31
32 void heapSort(int arr[], int n, int &comparisons) {
33     for (int i = n / 2 - 1; i >= 0; i--) heapify(arr, n, i, comparisons);
34     for (int end = n - 1; end > 0; end--) {
35         swap(arr[0], arr[end]);
36         heapify(arr, end, 0, comparisons);
37     }
38 }
39
40 #ifndef SORTING_TEST
41 int main() {
42     // A fixed seed lets all four programs use the same random arrays.
43     // Use srand(time(0)) with <ctime> if new arrays on every run are wanted.
44     srand(20261005);
```

heap_sort.cpp - continued

Source lines 45 through 87.

```

45     filesystem::create_directories("results");
46     ofstream summary("results/heap_sort_comparisons.csv");
47     ofstream samples("results/heap_sort_comparison_trials.csv");
48     ofstream output("results/heap_sort_output.txt");
49     if (!summary || !samples || !output) {
50         cerr << "Cannot open result files" << endl;
51         return 1;
52     }
53     summary << fixed << setprecision(6);
54     output << fixed << setprecision(2);
55     cout << fixed << setprecision(2);
56     summary << "n, trials, average_comparisons, n_log2_n, n_log10_n\n";
57     samples << "n, trial, comparisons\n";
58
59     for (int size = 30; size <= 1000; size += 10) {
60         int totalComparisons = 0;
61         for (int instance = 0; instance < 10; instance++) {
62             // Standard C++ fixed-capacity arrays; only the first size values are used.
63             int arr[1000], expected[1000];
64             for (int i = 0; i < size; i++) {
65                 arr[i] = rand() % 1000;
66                 expected[i] = arr[i];
67             }
68             // std::sort is only a reference to check the lab algorithm's output.
69             sort(expected, expected + size);
70             int comparisons = 0;
71             heapSort(arr, size, comparisons);
72             if (!equal(arr, arr + size, expected)) {
73                 cerr << "Incorrect sorted output" << endl;
74                 return 1;
75             }
76             totalComparisons += comparisons;
77             samples << size << ',' << instance + 1 << ',' << comparisons << '\n';
78         }
79         double averageComparisons = static_cast<double>(totalComparisons) / 10.0;
80         cout << size << " " << averageComparisons << endl;
81         output << size << " " << averageComparisons << '\n';
82         summary << size << ",10," << averageComparisons << ','
83             << size * log2(size) << ',' << size * log10(size) << '\n';
84     }
85     return 0;
86 }
87 #endif

```

insertion_sort_timing.cpp

Source lines 1 through 44.

```
1  #include <iostream>
2  #include <cstdlib>
3  #include <fstream>
4  #include <iomanip>
5  #include <chrono>
6  #include <filesystem>
7
8  using namespace std;
9
10 // Aim: Measure insertion sort for best, average and worst cases, n = 15000.
11 void insertionSortTimed(int arr[], int n) {
12     for (int i = 1; i < n; i++) {
13         int key = arr[i];
14         int j = i - 1;
15         while (j >= 0 && arr[j] > key) {
16             arr[j + 1] = arr[j];
17             j--;
18         }
19         arr[j + 1] = key;
20     }
21 }
22
23 // caseNumber: 0 = ascending, 1 = random permutation, 2 = descending.
24 void prepareInsertionInput(int arr[], int n, int caseNumber) {
25     for (int i = 0; i < n; i++) {
26         if (caseNumber == 2) arr[i] = n - i;
27         else arr[i] = i + 1;
28     }
29     if (caseNumber == 1) {
30         // Fisher-Yates shuffle gives distinct elements in a random order.
31         for (int i = n - 1; i > 0; i--) {
32             int j = rand() % (i + 1);
33             int temp = arr[i];
34             arr[i] = arr[j];
35             arr[j] = temp;
36         }
37     }
38 }
39
40 #ifndef SORTING_TEST
41 int main() {
42     const int n = 15000, trials = 5;
43     int arr[n]; // n is a compile-time constant, so this is standard C++.
44     const char *caseNames[] = {"best", "average", "worst"};
```

insertion_sort_timing.cpp - continued

Source lines 45 through 87.

```

45     srand(20261005);
46     filesystem::create_directories("results");
47     ofstream samples("results/insertion_sort_timing_trials.csv");
48     ofstream summary("results/insertion_sort_timing.csv");
49     if (!samples || !summary) {
50         cerr << "Cannot open result files" << endl;
51         return 1;
52     }
53     samples << fixed << setprecision(6);
54     summary << fixed << setprecision(6);
55     cout << fixed << setprecision(6);
56     samples << "case,n,trial,milliseconds\n";
57     summary << "case,n,trials,average_milliseconds\n";
58     cout << "Insertion sort: " << n << " elements, " << trials << " trials\n";
59     for (int caseNumber = 0; caseNumber < 3; caseNumber++) {
60         prepareInsertionInput(arr, n, caseNumber);
61         insertionSortTimed(arr, n); // One warmup per case.
62         double totalMilliseconds = 0;
63         for (int trial = 1; trial <= trials; trial++) {
64             prepareInsertionInput(arr, n, caseNumber);
65             auto start = chrono::steady_clock::now();
66             insertionSortTimed(arr, n);
67             auto end = chrono::steady_clock::now();
68             double milliseconds = chrono::duration<double, milli>(end - start).count();
69             // The input is a permutation of 1..n: verify every sorted value.
70             for (int i = 0; i < n; i++) {
71                 if (arr[i] != i + 1) {
72                     cerr << "Incorrect sorted output" << endl;
73                     return 1;
74                 }
75             }
76             totalMilliseconds += milliseconds;
77             samples << caseNames[caseNumber] << ',' << n << ',' << trial
78                 << ',' << milliseconds << '\n';
79         }
80         double averageMilliseconds = totalMilliseconds / trials;
81         summary << caseNames[caseNumber] << ',' << n << ',' << trials
82             << ',' << averageMilliseconds << '\n';
83         cout << caseNames[caseNumber] << " case: " << averageMilliseconds << " ms\n";
84     }
85     return 0;
86 }
87 #endif

```

radix_sort.cpp

Source lines 1 through 49.

```

1  #include <iostream>
2  #include <fstream>
3  #include <iomanip>
4  #include <algorithm>
5  #include <chrono>
6  #include <filesystem>
7  #include <cmath>
8
9  using namespace std;
10
11 // Aim: Sort nonnegative integers using LSD radix sort.
12 // Method: Apply stable counting sort to units, tens, hundreds, ... digits.
13 void sortByDigit(int arr[], int n, long long exponent) {
14     int count[10] = {0};
15     int *output = new int[n];
16     for (int i = 0; i < n; i++) count[(arr[i] / exponent) % 10]++;
17     for (int digit = 1; digit < 10; digit++) count[digit] += count[digit - 1];
18     // Traversing backwards preserves the order of equal digits (stability).
19     for (int i = n - 1; i >= 0; i--) {
20         int digit = static_cast<int>((arr[i] / exponent) % 10);
21         output[count[digit] - 1] = arr[i];
22         count[digit]--;
23     }
24     for (int i = 0; i < n; i++) arr[i] = output[i];
25     delete[] output;
26 }
27
28 bool radixSort(int arr[], int n) {
29     if (n < 0) return false;
30     if (n == 0) return true;
31     int maximum = arr[0];
32     for (int i = 0; i < n; i++) {
33         if (arr[i] < 0) return false;
34         if (arr[i] > maximum) maximum = arr[i];
35     }
36     // long long prevents overflow when exponent passes INT_MAX's last digit.
37     for (long long exponent = 1; maximum / exponent > 0; exponent *= 10)
38         sortByDigit(arr, n, exponent);
39     return true;
40 }
41
42 #ifndef SORTING_TEST
43 int main() {
44     const int n = 25, trials = 1000;
45     int input[n] = {170, 45, 75, 90, 802, 24, 2, 66, 0, 999,
46         501, 18, 39, 120, 300, 5, 75, 42, 610, 7, 88, 456, 321, 11, 200};
47     int expected[n], arr[n];
48     for (int i = 0; i < n; i++) expected[i] = input[i];
49     sort(expected, expected + n); // Reference only; never substitutes for our sort.

```

radix_sort.cpp - continued

Source lines 50 through 97.

```

50
51     cout << "Input (" << n << " elements): ";
52     for (int i = 0; i < n; i++) cout << input[i] << ' ';
53     cout << '\n';
54     // Warm up once before collecting measurements.
55     for (int i = 0; i < n; i++) arr[i] = input[i];
56     if (!radixSort(arr, n) || !equal(arr, arr + n, expected)) {
57         cerr << "Invalid input or incorrect sorted output" << endl;
58         return 1;
59     }
60     cout << "Sorted: ";
61     for (int i = 0; i < n; i++) cout << arr[i] << ' ';
62     cout << '\n';
63
64     filesystem::create_directories("results");
65     ofstream samples("results/radix_sort_timing_trials.csv");
66     ofstream summary("results/radix_sort_timing.csv");
67     if (!samples || !summary) {
68         cerr << "Cannot open result files" << endl;
69         return 1;
70     }
71     samples << fixed << setprecision(6);
72     summary << fixed << setprecision(6);
73     samples << "algorithm,n,trial,milliseconds\n";
74     double totalMilliseconds = 0;
75     for (int trial = 1; trial <= trials; trial++) {
76         for (int i = 0; i < n; i++) arr[i] = input[i];
77         auto start = chrono::steady_clock::now();
78         bool success = radixSort(arr, n);
79         auto end = chrono::steady_clock::now();
80         double milliseconds = chrono::duration<double, milli>(end - start).count();
81         // Copying, printing and verification are outside the timed section.
82         if (!success || !equal(arr, arr + n, expected)) {
83             cerr << "Incorrect sorted output" << endl;
84             return 1;
85         }
86         totalMilliseconds += milliseconds;
87         samples << "radix_sort," << n << ',' << trial << ',' << milliseconds << '\n';
88     }
89     double averageMilliseconds = totalMilliseconds / trials;
90     summary << "algorithm,n,trials,average_milliseconds\n"
91             << "radix_sort," << n << ',' << trials << ',' << averageMilliseconds << '\n';
92     cout << fixed << setprecision(6)
93          << "Average execution time over " << trials << " trials: "
94          << averageMilliseconds << " ms" << endl;
95     return 0;
96 }
97 #endif

```

bucket_sort.cpp

Source lines 1 through 52.

```

1  #include <iostream>
2  #include <fstream>
3  #include <iomanip>
4  #include <algorithm>
5  #include <chrono>
6  #include <filesystem>
7  #include <cmath>
8  #include <vector>
9
10 using namespace std;
11
12 // Aim: Sort fractions in [0, 1) using textbook bucket sort.
13 void insertionSortBucket(vector<double> &bucket) {
14     for (int i = 1; i < static_cast<int>(bucket.size()); i++) {
15         double key = bucket[i];
16         int j = i - 1;
17         while (j >= 0 && bucket[j] > key) {
18             bucket[j + 1] = bucket[j];
19             j--;
20         }
21         bucket[j + 1] = key;
22     }
23 }
24
25 bool bucketSort(double arr[], int n) {
26     if (n < 0) return false;
27     if (n == 0) return true;
28     for (int i = 0; i < n; i++)
29         if (!isfinite(arr[i]) || arr[i] < 0.0 || arr[i] >= 1.0) return false;
30
31     // Create n bucket lists; vector stores a variable number of values per list.
32     vector<double> *buckets = new vector<double>[n];
33     for (int i = 0; i < n; i++) {
34         int index = static_cast<int>(n * arr[i]);
35         if (index >= n) index = n - 1;
36         buckets[index].push_back(arr[i]);
37     }
38     int next = 0;
39     for (int i = 0; i < n; i++) {
40         insertionSortBucket(buckets[i]);
41         for (int j = 0; j < static_cast<int>(buckets[i].size()); j++)
42             arr[next++] = buckets[i][j];
43     }
44     delete[] buckets;
45     return true;
46 }
47
48 #ifndef SORTING_TEST
49 int main() {
50     const int n = 25, trials = 1000;
51     double input[n] = {0.170, 0.045, 0.075, 0.090, 0.802, 0.024, 0.002, 0.066, 0.000,
52         0.999,
53         0.501, 0.018, 0.039, 0.120, 0.300, 0.005, 0.075, 0.042, 0.610, 0.007,

```

bucket_sort.cpp - continued

Source lines 53 through 104.

```

53         0.088, 0.456, 0.321, 0.011, 0.200};
54     double expected[n], arr[n];
55     for (int i = 0; i < n; i++) expected[i] = input[i];
56     sort(expected, expected + n); // Reference only; never substitutes for our sort.
57
58     cout << "Input (" << n << " elements): ";
59     for (int i = 0; i < n; i++) cout << input[i] << ' ';
60     cout << '\n';
61     // Warm up once before collecting measurements.
62     for (int i = 0; i < n; i++) arr[i] = input[i];
63     if (!bucketSort(arr, n) || !equal(arr, arr + n, expected)) {
64         cerr << "Invalid input or incorrect sorted output" << endl;
65         return 1;
66     }
67     cout << "Sorted: ";
68     for (int i = 0; i < n; i++) cout << arr[i] << ' ';
69     cout << '\n';
70
71     filesystem::create_directories("results");
72     ofstream samples("results/bucket_sort_timing_trials.csv");
73     ofstream summary("results/bucket_sort_timing.csv");
74     if (!samples || !summary) {
75         cerr << "Cannot open result files" << endl;
76         return 1;
77     }
78     samples << fixed << setprecision(6);
79     summary << fixed << setprecision(6);
80     samples << "algorithm,n,trial,milliseconds\n";
81     double totalMilliseconds = 0;
82     for (int trial = 1; trial <= trials; trial++) {
83         for (int i = 0; i < n; i++) arr[i] = input[i];
84         auto start = chrono::steady_clock::now();
85         bool success = bucketSort(arr, n);
86         auto end = chrono::steady_clock::now();
87         double milliseconds = chrono::duration<double, milli>(end - start).count();
88         // Copying, printing and verification are outside the timed section.
89         if (!success || !equal(arr, arr + n, expected)) {
90             cerr << "Incorrect sorted output" << endl;
91             return 1;
92         }
93         totalMilliseconds += milliseconds;
94         samples << "bucket_sort," << n << ',' << trial << ',' << milliseconds << '\n';
95     }
96     double averageMilliseconds = totalMilliseconds / trials;
97     summary << "algorithm,n,trials,average_milliseconds\n"
98         << "bucket_sort," << n << ',' << trials << ',' << averageMilliseconds <<
99     '\n';
100     cout << fixed << setprecision(6)
101         << "Average execution time over " << trials << " trials: "
102         << averageMilliseconds << " ms" << endl;
103     return 0;
104 }
#endif

```

counting_sort.cpp

Source lines 1 through 46.

```

1  #include <iostream>
2  #include <fstream>
3  #include <iomanip>
4  #include <algorithm>
5  #include <chrono>
6  #include <filesystem>
7  #include <cmath>
8
9  using namespace std;
10
11 // Aim: Sort nonnegative integers using stable counting sort.
12 // Method: Count frequencies, form prefix sums, and place elements backwards.
13 bool countingSort(int arr[], int n) {
14     if (n < 0) return false;
15     if (n == 0) return true;
16     int maximum = arr[0];
17     for (int i = 0; i < n; i++) {
18         // Keep the frequency table within a reasonable memory limit.
19         if (arr[i] < 0 || arr[i] > 1000000) return false;
20         if (arr[i] > maximum) maximum = arr[i];
21     }
22     int *count = new int[maximum + 1](); // Initialize every frequency to zero.
23     int *output = new int[n];
24     for (int i = 0; i < n; i++) count[arr[i]]++;
25     for (int value = 1; value <= maximum; value++) count[value] += count[value - 1];
26     for (int i = n - 1; i >= 0; i--) {
27         output[count[arr[i]] - 1] = arr[i];
28         count[arr[i]]--;
29     }
30     for (int i = 0; i < n; i++) arr[i] = output[i];
31     delete[] count;
32     delete[] output;
33     return true;
34 }
35
36 #ifndef SORTING_TEST
37 int main() {
38     const int n = 25, trials = 1000;
39     int input[n] = {170, 45, 75, 90, 802, 24, 2, 66, 0, 999,
40         501, 18, 39, 120, 300, 5, 75, 42, 610, 7, 88, 456, 321, 11, 200};
41     int expected[n], arr[n];
42     for (int i = 0; i < n; i++) expected[i] = input[i];
43     sort(expected, expected + n); // Reference only; never substitutes for our sort.
44
45     cout << "Input (" << n << " elements): ";
46     for (int i = 0; i < n; i++) cout << input[i] << ' ';

```

counting_sort.cpp - continued

Source lines 47 through 91.

```

47     cout << '\n';
48     // Warm up once before collecting measurements.
49     for (int i = 0; i < n; i++) arr[i] = input[i];
50     if (!countingSort(arr, n) || !equal(arr, arr + n, expected)) {
51         cerr << "Invalid input or incorrect sorted output" << endl;
52         return 1;
53     }
54     cout << "Sorted: ";
55     for (int i = 0; i < n; i++) cout << arr[i] << ' ';
56     cout << '\n';
57
58     filesystem::create_directories("results");
59     ofstream samples("results/counting_sort_timing_trials.csv");
60     ofstream summary("results/counting_sort_timing.csv");
61     if (!samples || !summary) {
62         cerr << "Cannot open result files" << endl;
63         return 1;
64     }
65     samples << fixed << setprecision(6);
66     summary << fixed << setprecision(6);
67     samples << "algorithm,n,trial,milliseconds\n";
68     double totalMilliseconds = 0;
69     for (int trial = 1; trial <= trials; trial++) {
70         for (int i = 0; i < n; i++) arr[i] = input[i];
71         auto start = chrono::steady_clock::now();
72         bool success = countingSort(arr, n);
73         auto end = chrono::steady_clock::now();
74         double milliseconds = chrono::duration<double, milli>(end - start).count();
75         // Copying, printing and verification are outside the timed section.
76         if (!success || !equal(arr, arr + n, expected)) {
77             cerr << "Incorrect sorted output" << endl;
78             return 1;
79         }
80         totalMilliseconds += milliseconds;
81         samples << "counting_sort," << n << ',' << trial << ',' << milliseconds << '\n';
82     }
83     double averageMilliseconds = totalMilliseconds / trials;
84     summary << "algorithm,n,trials,average_milliseconds\n"
85             << "counting_sort," << n << ',' << trials << ',' << averageMilliseconds <<
86     '\n';
87     cout << fixed << setprecision(6)
88         << "Average execution time over " << trials << " trials: "
89         << averageMilliseconds << " ms" << endl;
90     return 0;
91 }
92 #endif

```

Appendix B. Sources and measurement notes

Implementation and experiment authority

GitHub repository: [im-lakshyaveerturna/sorting-algorithms-cpp](https://github.com/im-lakshyaveerturna/sorting-algorithms-cpp)

Explanations and complete listings correspond to commit 14b5fd3ed5f8bd3963532d16f0a86ce77795ab37. Program behavior, counters and measured results are grounded in those source files and the saved raw/summary CSVs. The user-supplied INSERTION.CPP is the authority for the size range, ten-instance structure and insertion shift counter.

Teaching references

IIT Ropar: Lecture 18, Sorting. Covers recursive merge sort, last-pivot partitioning, counting sort, bucket sort and radix sort. [Open the lecture slides.](#)

NPTEL / IIT Guwahati: Data Structures, Sorting module. Introduces comparison-based sorting methods and their analysis. [Open the module.](#)

NPTEL / IIT Madras: Programming, Data Structures and Algorithms using C, course 106106127. Course topics include insertion, merge, counting and radix sorting. [Open the course page.](#)

These references support the conventional undergraduate algorithms. Individual colleges may prescribe a different partition scheme, language version, counting convention or observation-table format. This guide uses the variants and experiment format actually submitted for this assignment.

Measurement environment

Item	Recorded setting
Date	05 October 2026
Platform	macOS 26.2, ARM64
Compiler	Apple Clang 17.0.0
Flags	-std=c++17 -O2 -Wall -Wextra -Wpedantic
Random seed	20261005; sequences depend on C-library implementation
Comparison cases	98 sizes, ten instances each, values <code>rand() % 1000</code>
Insertion timing	15,000 values, five measured trials per case
Other timing tasks	25 values, 1,000 measured trials each

Saved means were checked against raw trial files, and both logarithmic reference formulas were checked. Small timings retain timer overhead; they should not be used as a universal ranking. No production deployment or external benchmark claim is implied by this classroom experiment.